The Essential of Chapter 1

DBMS internals in four pages: which subsystem, which stage, who chooses the plan, how a plan tree runs, the SQL you must write

Advanced Data Management

2026-10-31

Numbers to carry: `customers` 10,000 rows / 73 pages; `orders` 500,000 rows / 3,185 pages; 8 KB pages. Chapter 1 is worth about 20 points over the two mocks (v2 A1, C1; Mock 1 2.1, 3.7), but its words (planner, binding, build side, blocking) appear in every other question.

1 Five subsystems, and who owns a symptom

Concept: The declarative gap

SQL says *what* result you want, nothing about *how*. The DBMS must choose the **plan**: scan or index, which join algorithm, in which order. Every mechanism in the course exists to make that choice well.

1. **Connection/session manager**: authenticates, one worker process per connection. No database work.
2. **Query processor**: **parser** (grammar), **optimiser** = PostgreSQL's *planner* (cheapest plan), **executor** (runs it).
3. **Storage manager**: pages and files (ch. 2), access methods = indexes (ch. 3), **buffer pool** = cache of disk pages (ch. 6).
4. **Transaction manager**: lock manager (concurrency), log, recovery (all of a transaction or none).
5. **Catalog**: tables, columns, indexes and the **statistics** the planner lives on (`pg_class`, `pg_stats`).

Symptom → owner. Seq Scan where an index would be faster: planner. Two updates, one lost: transaction manager (locks). Low hit ratio: buffer pool. "column `amuont` does not exist": binder, reading the catalog. Estimate off by 1,000×: stale statistics, `ANALYZE`.

Why the DBMS keeps its own buffer pool instead of the OS cache (three reasons, each "the DBMS knows X, the OS cannot, so Y breaks"): (1) the *access pattern*: a scan touches each page once; the OS's generic LRU lets one big scan evict every hot page (sequential flooding); (2) the *write order*: the log record must hit disk *before* the data page it describes; the OS writes

dirty pages in any order, so a crash can leave a modified page with no record to undo it; (3) *pinning and memory budget*: a page in use must not be paged out, memory is budgeted per operator (`work_mem`), and caching a page twice is double buffering.

Mock drill

Mock 1, 2.1 (7 pts), **Quiz C6(a)**: the three reasons above, numbered; the rubric wants the durability one by name. **Quiz A1, C5(e)**: a lost update belongs to the transaction manager, not storage.

2 The life of a query: five stages, five objects

Stage	Input → output	What it checks or decides	Typical error here
1 Parse	SQL text → syntax tree	grammar only	`FORM` instead of `FROM`
2 Bind	syntax tree → validated query	names exist, types, permissions, **against the catalog**	"relation `ordrs` does not exist"
3 Rewrite	validated query → logical plan	relational algebra; views expanded; **predicate pushdown**	—
4 Optimise	logical → physical plan	access path, join algorithm, join order, **by cost from statistics**	bad plan from stale stats
5 Execute	physical plan → rows	iterator model	—

Logical plan of the running query

```
SELECT c.name, SUM(o.amount) FROM customers c JOIN orders o ON o.
    cust_id = c.id
WHERE c.country = 'MA' GROUP BY c.name;
```

bottom-up: $\sigma_{\text{country='MA'}}(\text{customers})$, $\bowtie_{\text{cust_id=id}}$ orders, $\Gamma_{\text{name; SUM(amount)}}$, $\pi_{\text{name,total}}$. The filter sits *below* the join although the SQL wrote it after: **predicate pushdown**, safe whenever the predicate mentions one input's columns only, good because every operator above does less work.

The optimiser's three decisions: access path (Seq Scan at 1 per page against Index Scan at a random page, cost 4, per matching row: the index pays only when few rows match), join algorithm (nested loop / sort-merge / hash), order. Many physical plans for one logical plan; the same SQL gets a different plan when data grows or an index appears.

Key insight. "Does not exist" is always a *binding* error: the parser cannot know, only the catalog can. Seq Scan versus Index Scan is always the *planner's* choice: the executor runs what it is given, the buffer manager never sees the SQL.

Mock drill

Quiz A2, A3, B5, C5(d): `SELECT * FROM ordrs` fails at binding (b). **Quiz A9, C5(b)**: pushdown happens at the rewrite stage. **Mock v2 A1** (3 pts): the planner (b); rule out parser, executor, buffer manager in one clause each. **Quiz C5(c)**: estimate 4 rows against 9,000 actual = stale statistics, `ANALYZE events`.

3 Execution: iterators, streaming and blocking

Mechanism: Iterator (Volcano) model

Every operator has `open()`, `next()`, `close()`. The root is asked for a tuple and asks its children: **requests flow down, tuples flow up, one at a time** (pipelining: the join produces while the scan still reads). **Streaming** operators emit after seeing part of their input: scan with filter, projection, the *probe* side of a hash join. **Blocking** operators (pipeline breakers) must consume their whole input first: Sort, HashAggregate, and the **build side** of a hash join (the `Hash` node).

Reading EXPLAIN: the deepest-indented leaf runs first, so read bottom-up. Under a `Hash Join`, the child under the `Hash` node is the build side (read entirely into memory first); the other child is the probe side (streamed). Mock v2 Part C:

```
Sort (rows=5)  Sort Key: (sum(o.amount)) DESC                    blocking
  -> HashAggregate (rows=5)  Group Key: c.country                 blocking
     -> Hash Join  Hash Cond: (o.cust_id = c.id)                  streams the probe
     side
        -> Seq Scan on orders o  Filter: (order_date >= ...)      streaming (42,299
     kept)
        -> Hash  (Batches: 1  Memory Usage: 519kB)                 blocking: build
     side
           -> Seq Scan on customers c (rows=10000)                streaming
```

Mock drill

Mock v2 C1 (5 pts): draw the five nodes, label each with the reason in half a sentence as above, close with "two of five stream; the first row appears only at the end". **Quiz A5–A8, C5(a)**: open/next/close; what breaks the pipeline; the child under `Hash` is the build side.

4 The SQL the exam makes you write

SQL is written SELECT-first but **evaluated FROM-first**: (1) FROM/JOIN (2) WHERE (3) GROUP BY (4) HAVING (5) SELECT, aliases are born here (6) ORDER BY. So: an alias **cannot** be used in WHERE but can in ORDER BY; WHERE filters rows and may not hold an aggregate; HAVING filters groups (`COUNT(*) > 100` goes there); every selected column is in GROUP BY or inside an aggregate. The order 1–6 is the logical plan read bottom-up ($\bowtie$, σ, Γ, σ, π).

```sql
SELECT   u.country, count(*) AS purchases              -- Mock 1,
    3.7(a); exam date 2026-10-12
FROM     events e JOIN users u ON u.user_id = e.user_id
WHERE    e.event_type = 'purchase' AND e.ts >= date '2026-10-12' -
    interval '30 days'
GROUP BY u.country
ORDER BY purchases DESC;
```

Mock drill

Mock 1, 3.7(a): points for the join condition, both filters in WHERE (not HAVING), GROUP BY on the non-aggregated column, a sensible ORDER BY. Both filters mention only `events` columns, so the rewriter pushes them below the join.

5 OLTP versus OLAP, rows versus columns

OLTP: many short transactions touching a few whole rows; **row store** (all fields of a record together, one page per record access). **OLAP**: few long queries scanning millions of rows but few columns; **column store** (one column's values together: read only that column's pages, and same-type runs compress well). With 30 similar columns, `SELECT avg(price)` reads every page of a row store but about 1/30 of a column store's, before compression. A single-row update in a column store touches one page per column instead of one page.

Mock drill

Quiz A10, B6: 1/30 (b), one line why. **Mock 1, 3.2(b)**: the row scan reads 8 GB (every column of every row) for one 8-byte `ts`; Parquet holds that column alone, 50M $\times$ 8 = 400 MB raw, tens of MB compressed.

Warning / Gotcha: The four mistakes that cost points

- "The parser catches a misspelled column." No: the binder does, against the catalog.
- "The executor or buffer manager picks Seq Scan versus Index Scan." The planner does, by cost from statistics.
- Labelling the Hash Join blocking. The *build side* (the `Hash` node) blocks; the join streams the probe side.
- An alias or an aggregate in WHERE. Aliases go in ORDER BY, aggregates in HAVING.

Summary: The essential in four lines

1. SQL says what, the DBMS chooses how. Five subsystems: connection, query processor (parser, planner, executor), storage (pages, indexes, buffer pool), transactions (locks, log, recovery), catalog.
2. Parse (grammar) $\rightarrow$ bind (catalog) $\rightarrow$ rewrite (algebra, pushdown) $\rightarrow$ optimise (access path, join algorithm, order, by cost: 1 sequential, 4 random) $\rightarrow$ execute.
3. Iterators: requests down, tuples up. Streaming: scans, probe side. Blocking: Sort, HashAggregate, Hash build. Read EXPLAIN bottom-up.
4. FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY. OLTP = rows = row store; OLAP = few columns = column store.